

Slime Rush - 마법 생성 및 자동 타겟팅

스킬 인스턴스 생성부터 타겟 검색까지

플레이어에게 할당된 마법 정보를 수신하고, 실행 가능한 인스턴스로 생성한 뒤, 쿨타임과 자동 캐스팅 루틴을 관리하면서 타겟팅 구조체 기준으로 타겟을 검색하는 흐름을 정리한 코드 샘플입니다.

MagicBookLibrary

MagicBookCreator

MagicBook

TargetingOption

TargetSystem

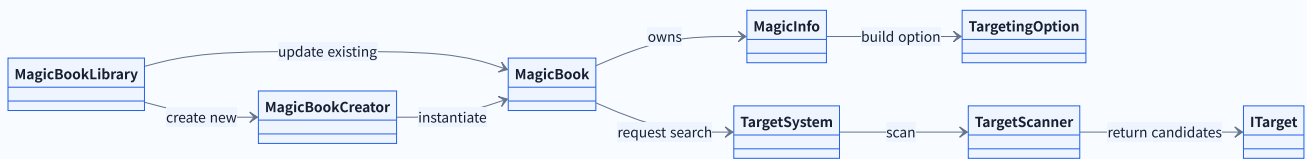
TargetScanner

샘플 목표

- 마법 할당 이벤트를 받아 마법 인스턴스 생성 또는 업데이트
- 데이터를 통한 마법 인스턴스 초기화
- 타겟팅 옵션의 생성
- 쿨타임마다 타겟/위치 검색
- 검색 결과를 통해 마법 투사체 캐스팅까지 연결

설계 기준

- **MagicBookLibrary**: 마법 할당 이벤트 수신과 생성/업데이트
- **MagicBookCreator**: MagicBook 생성 및 초기화
- **MagicBook**: 쿨타임, 타겟팅 옵션, 자동 검색 루틴 관리
- **TargetSystem**: 타겟 타입에 맞는 타겟/위치 검색 진입점
- **TargetScanner**: 거리, 랜덤, 개수 제한 기반 후보 탐색



프로젝트	Slime Rush: Leshy and the Magic Book
해당 영역	BattleSystem, TargetSystem
샘플 범위	마법 할당, 마법 인스턴스 생성/업데이트, 타겟옵션 구성, 자동 검색 루틴, 타겟 후보 탐색
공개 방식	실제 흐름은 유지하되 외부 생성 도구, 데미지/이펙트 등 일부 구현 세부사항은 제거해 채용 검토용으로 축약
제작 영상	제작영상1 제작영상2 제작영상3

마법 데이터와 타겟팅 옵션

```
public enum MagicTargetType
{
    None,
    Mouse,
    Self,
    Nearest,
    RandomTarget
}

public struct MagicInfo
{
    public string groupId;
    public MagicTargetType targetingType;
    public int hitCount;
    public int attackRepeatCount;
    public float attackRange;
    ...
}

public struct TargetingOption
{
    public MagicTargetType targetType;
    public int targetCount;
    public float targetingRange;

    public TargetingOption(
        MagicTargetType targetType,
        int targetCount,
        float targetingRange)
    {
        TargetType = targetType;
        TargetCount = targetCount;
        TargetingRange = targetingRange;
    }
}
```

실무 의도

- MagicInfo는 마법의 기본 데이터이고, TargetingOption은 자동 캐스팅 루틴에서 필요한 타겟 타입, 타겟 수, 탐색 범위 정의
- 실제 프로젝트의 쿨타임, 리소스 ID, 속성, 계수 등 더 많은 리소스 정보가 포함되어 있으나, 본 샘플에서는 타겟팅 흐름을 설명하는 데 필요한 필드만 공개용으로 축약

MagicBook 생성과 업데이트

```
public sealed class MagicBookLibrary : MonoBehaviour
{
    private MagicBookCreator creator;
    private MagicBook prefab;
    private List<MagicBook> magicBooks = new();

    public void OnUpdatedMagic(MagicInfo magicInfo)
    {
        var book = magicBooks.Find(x => x.GetMagicGroupId == magicInfo.groupId);

        if (book != null)
        {
            book.OnUpdatedMagicInfo(magicInfo);
            return;
        }

        StartCoroutine(CreateMagicBook(magicInfo));
    }

    private IEnumerator CreateMagicBook(MagicInfo magicInfo)
    {
        yield return new WaitUntil(() => playerInitialized);

        var book = creator.Create(this, prefab, magicInfo, playerInfo);
        magicBooks.Add(book);
    }
}
```

실무 의도

- MagicBookLibrary는 활성화된 MagicBook 목록만 관리
- 같은 groupId의 마법은 새로 만들지 않고 기존 MagicBook을 업데이트하고, 신규 마법만 생성

MagicBook 생성과 초기화

```
public sealed class MagicBookCreator
{
    public MagicBook Create(
        MagicBookLibrary parent,
        MagicBook prefab,
        MagicInfo magicInfo,
        PlayerDataInfo playerInfo)
    {
        if (parent == null)
            return null;

        var book = UnityEngine.Object.Instantiate(
            prefab,
            parent.transform);

        book.InitializeMagicBook(magicInfo, playerInfo);
        return book;
    }
}
```

실무 의도

- 원본은 외부 DI/Factory 도구를 사용하지만, 샘플에서는 생성 흐름만 보이도록 Unity 기본 생성 형태로 정리
- MagicInfo와 PlayerDataInfo가 MagicBook.InitializeMagicBook로 전달

초기화와 타겟팅 프로세스 구성

```
public sealed class MagicBook : MonoBehaviour
{
    private const float TargetSearchDelay = 0.1f;

    private TargetSystem targetSystem;
    private MagicInfo magicInfo;
    private PlayerDataInfo playerInfo;
    private ITarget player;
    private TargetingOption targetingOption;
    private IEnumerator targetSearcher;
    private Coroutine searchRoutine;
    private float calculatedCoolTime;

    public string GetMagicGroupId => magicInfo.groupId;

    public void InitializeMagicBook(MagicInfo info, PlayerDataInfo playerData)
    {
        magicInfo = info;
        playerInfo = playerData;
        player = targetSystem.PlayerTarget;
        var targetCount = Mathf.Max(info.hitCount, info.attackRepeatCount);
        targetingOption = new TargetingOption(info.targetingType, targetCount, info.attackRange);
        UpdateMagicBook();
    }

    public void OnUpdatedMagicInfo(MagicInfo info)
    {
        magicInfo = info;
        UpdateMagicBook();
    }

    private void UpdateMagicBook()
    {
        if (searchRoutine != null)
            StopCoroutine(searchRoutine);

        targetSearcher = SetTargetingProcess(targetingOption);
        searchRoutine = StartCoroutine(targetSearcher);

        calculatedCoolTime = CommonBattleManager.GetCoolTime(
            magicInfo.attackSpeed,
            playerInfo.AttackSpeed);
    }
}
```

실무 의도

- MagicBook은 생성된 뒤 마법 데이터와 플레이어 데이터를 받아 타겟팅 옵션을 만들고, 자동 검색 루틴을 갱신

타겟 타입에 따른 검색/캐스팅 경로 선택

```
private IEnumerator SetTargetingProcess(TargetingOption option)
{
    if (option.TargetType == MagicTargetType.Self)
        return CoSearchSelf(CastToTarget);

    var isSingleTarget = option.TargetCount <= 1;

    if (option.TargetType is MagicTargetType.None or MagicTargetType.Mouse)
    {
        return CoSearch(CastToPosition, type =>
            isSingleTarget
                ? new[] { targetSystem.FindPosition(player.GetPosition(), type, option.TargetingRange) }
                : targetSystem.FindPositions(player.GetPosition(), type, option.TargetCount, option.TargetingRange));
    }
    if (option.TargetType is MagicTargetType.Nearest or MagicTargetType.RandomTarget)
    {
        return CoSearch(CastToTarget, type =>
            isSingleTarget
                ? new[] { targetSystem.FindTarget(player.GetPosition(), type, option.TargetingRange) }
                : targetSystem.FindTargets(player.GetPosition(), type, option.TargetCount, option.TargetingRange));
    }
    return null;
}

private IEnumerator CoSearchSelf(Action<ITarget[]> castAction)
{
    return CoSearch(castAction, _ => new[] { player });
}
```

실무 의도

- SetTargetingProcess는 타겟 타입과 타겟 수만 보고 위치 공격, 단일 타겟 공격, 다중 타겟 공격, 내 위치 공격 루틴을 선택

쿨타임 기반 자동 검색 루틴

```
private IEnumerator CoSearch<T>(
    Action<T> castAction,
    Func<MagicTargetType, T> findMethod)
{
    var remainCoolTime = calculatedCoolTime;
    var wait = new WaitForSeconds(TargetSearchDelay);

    while (true)
    {
        var result = findMethod(magicInfo.targetingType);

        if (result != null && remainCoolTime <= 0f)
        {
            castAction(result);
            remainCoolTime = calculatedCoolTime;
        }
        else
        {
            remainCoolTime = Mathf.Max(0f, remainCoolTime - TargetSearchDelay);
        }
        yield return wait;
    }
}
```

실무 의도

- MagicBook은 자체 코루틴으로 쿨타임을 확인하고, 준비되면 타겟 검색 결과를 캐스팅 함수에 전달

TargetSystem과 TargetScanner 최소 구현

```
public sealed class TargetSystem
{
    private readonly TargetScanner scanner;
    private readonly List<ITarget> enemies = new();
    public ITarget PlayerTarget { get; private set; }
    public ITarget FindTarget(Vector3 origin, MagicTargetType type, float range)
    {
        return FindTargets(origin, type, 1, range).FirstOrDefault();
    }

    public ITarget[] FindTargets(Vector3 origin, MagicTargetType type, int count, float range)
    {
        var candidates = enemies.Where(target => target.IsAlive);
        return type switch
        {
            MagicTargetType.Nearest => scanner.FindNearest(origin, candidates, count, range),
            MagicTargetType.RandomTarget => scanner.FindRandom(origin, candidates, count, range),
        };
    }

    public Vector3 FindPosition(Vector3 origin, MagicTargetType type, float range)
    {
        ...
    }
}

public sealed class TargetScanner
{
    public ITarget[] FindNearest(Vector3 origin, IEnumerable<ITarget> candidates, int count, float range)
    {
        return candidates.Where(target => IsInRange(origin, target, range))
            .OrderBy(target => DistanceSqr(origin, target))
            .Take(count)
            .ToArray();
    }

    public ITarget[] FindRandom(Vector3 origin, IEnumerable<ITarget> candidates, int count, float range)
    {
        return candidates.Where(target => IsInRange(origin, target, range))
            .OrderBy(_ => UnityEngine.Random.value)
            .Take(count)
            .ToArray();
    }

    private static bool IsInRange(Vector3 origin, ITarget target, float range)
    => range <= 0f || DistanceSqr(origin, target) <= range * range;

    private static float DistanceSqr(Vector3 origin, ITarget target)
    => (target.GetPosition() - origin).sqrMagnitude;
}
```

실무 의도

- 위치 기반 스킬의 FindPosition은 별도 입력/좌표 계산으로 샘플에서는 타겟 객체 탐색 흐름을 중심으로 설명하기 위해 제외
- MagicBook은 타겟팅 타입과 타겟 수를 기준으로 검색 요청만 만들고, 실제 후보 수집, 거리 비교, 랜덤 선택, 개수 제한은 TargetSystem과 TargetScanner로 분리

내용 정리

- MagicBookLibrary는 생성/업데이트 판단만 담당
- MagicBook은 쿨타임과 자동 캐스팅 루틴 담당
- TargetSystem은 타겟 타입을 해석하고 검색을 위임
- TargetScanner는 거리, 랜덤, 개수 제한을 처리